Fundamental Programming in JAVA

ONE-DIMENSIONAL ARRAYS

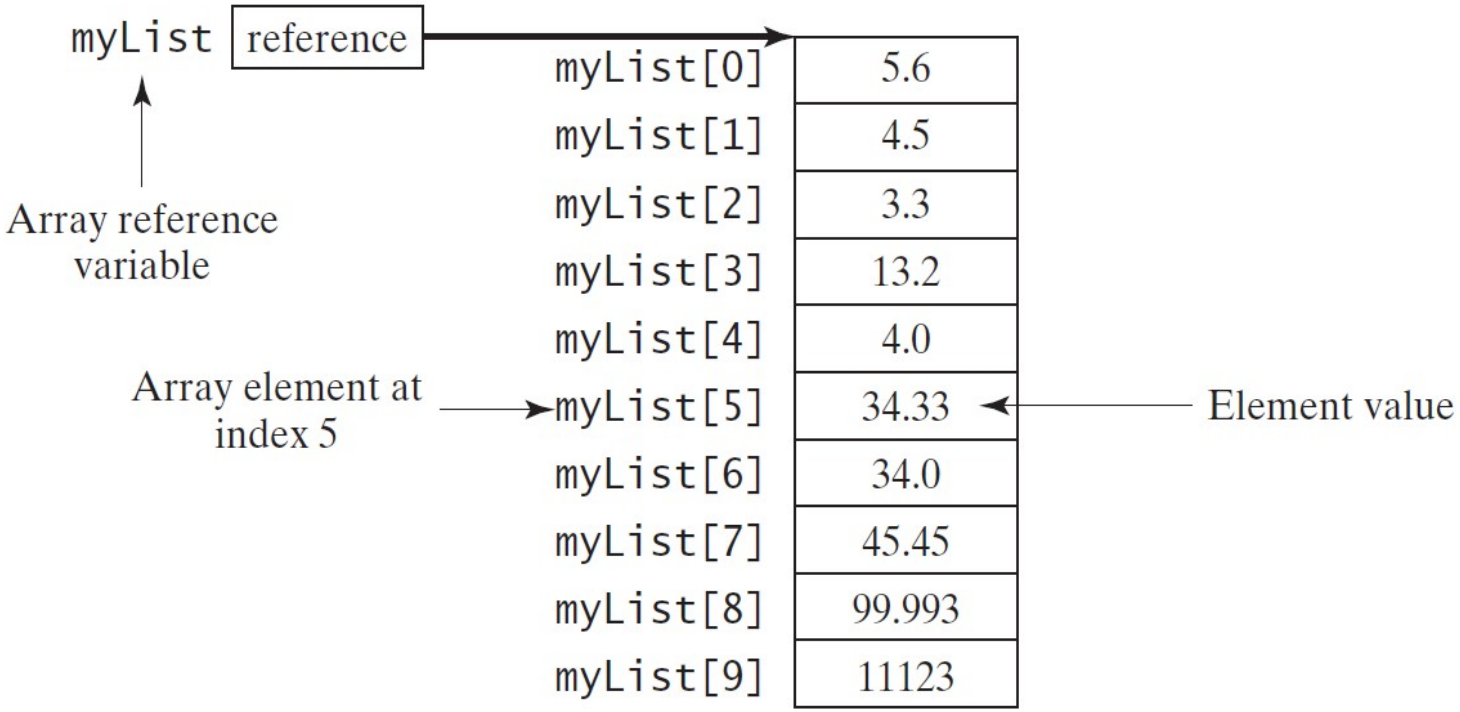
Objectives

- To declare array reference variables and create arrays.
- To obtain array size using arrayRefVar.length and know default values in an array.
- To access array elements using indexes.
- To declare, create, and initialize an array using an array initializer.
- To simplify programming using the foreach loops.
- To copy contents from one array to another.
- To program common array operations (displaying arrays, summing all elements and finding the minimum and maximum).

Introducing Arrays

- Array is a data structure that represents a collection of the same types of data.

```
double[] myList = new double[10];
```



Array Declaration

- The syntax for declaring an array variable is:

```
datatype[] arrayRefVar;
```

Example :

```
double[] myList;
```

Array creation

- The syntax for creating the array is:

```
arrayName = new DataType[arraySize];
```

- **Example** :

```
myList = new double[10];
```

Declaring and Creating in **One Step**

```
datatype[] arrayRefVar = new  datatype[arraySize];
```

- Examples:

```
double[] myList = new double[10];
double [ ] marks = new double [45];
int  number [ ] = new int [100];
```

Declaring, creating, initializing Using the Shorthand Notation

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

- This **shorthand** notation is equivalent to the following statements:

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```

CAUTION

- Using the shorthand notation, you have to declare, create, and initialize the array all in one statement. Splitting it would cause a syntax error. For example, the following is wrong:

```
double[] myList;

myList = {1.9, 2.9, 3.4, 3.5};
```

Alternate Array Declaration

- Previously we showed arrays being declared: `double[] myList;`

However, the brackets can also be after the array name: `double myList[];`

These are equivalent but the first style is typical.

- Multiple arrays can be declared on the same line.

```
double[] myList, codes, scores;
```

- With the alternate notation each variable must have brackets.

```
double myList[], codes[], scores;
```

  - The **scores** variable in this instance is **simply a *double* variable**.

Processing Array Contents

- Processing data in an array is the same as any other variable.

```
grossPay = hours[3] * payRate;
```

- Array elements can be used in relational operations:

```
if(cost[20] < cost[0]) {
//statements
}
```

- They can be used as loop conditions:

```
while(value[count] != 0) {
   //statements
}
```

The Length of an Array

- Once an array is created, its size is fixed. It cannot be changed. You can find its size using

  `arrayRefVar.length`

- For example,

  `myList.length` returns 10

Default Values

- When an array is created, its elements are assigned the default values as following:

Data type	Default value
byte, short, int ,long	0
float ,double	0.0
char	'\u0000'
boolean	false
String, reference to objects	null

Off-by-One Errors

- It is very easy to be off-by-one when accessing arrays.

```
// This code has an off-by-one error.
int[] numbers = new int[100];
for (int i = 1; i <= 100; i++)
  numbers[i] = 99;
```

- Here, the equal sign allows the loop to continue on to index 100, where 99 is the last index in the array.
- This code would throw an **`ArrayIndexOutOfBoundsException`**.

Array Program Tracing

```
public class Test {
  public static void main(String[]
    args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] +
    values[4];
  }
}
```

Declare array variable values, create an array, and assign its reference to values

After the array is created

0	0
1	0
2	0
3	0
4	0

Array Program Tracing

```
public class Test {

  public static void main(String[]
    args) {

    int[] values = new int[5];

    for (int i = 1; i < 5; i++) {

      values[i] = i + values[i-1];

    }

    values[0] = values[1] +
    values[4];

  }

}
```

i becomes 1

After the array is created

0	0
1	0
2	0
3	0
4	0

Array Program Tracing

```
public class Test {

  public static void main(String[]
    args) {

    int[] values = new int[5];

    for (int i = 1; i < 5; i++) {

      values[i] = i + values[i-1];

    }

    values[0] = values[1] +
    values[4];

  }

}
```

i is less than 5

After the array is created

0	0
1	0
2	0
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[]
    args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this line is executed, value[1] is 1

After the first iteration

0	0
1	1
2	0
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After i++, i becomes 2

After the first iteration

0	0
1	1
2	0
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[]
    args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

i is less than 5

After the first iteration

0	0
1	1
2	0
3	0
4	0

Array Program Tracing

```
public class Test {

  public static void main(String[]
    args) {

    int[] values = new int[5];

    for (int i = 1; i < 5; i++) {

      values[i] = i + values[i-1];

    }

    values[0] = values[1] +
    values[4];

  }

}
```

After this line, values[2] is 3 (2 + 1)

After the second iteration

0	0
1	1
2	3
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this, i becomes 3.

After the second iteration

0	0
1	1
2	3
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

i is still less than 5.

After the second iteration

0	0
1	1
2	3
3	0
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this line, values[3] becomes 6 (3 + 3)

After the third iteration

0	0
1	1
2	3
3	6
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this, i becomes 4

After the third iteration

0	0
1	1
2	3
3	6
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

I is still less than 5

After the third iteration

0	0
1	1
2	3
3	6
4	0

Array Program Tracing

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this, values[4] becomes 10 (4 + 6)

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After i++, i becomes 5

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Array Program Tracing

i < 5 is false. Exit the loop

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Array Program Tracing

```
public class Test {
  public static void main(String[] args)
    {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After this line, values[0] is 11 (1 + 10)

0	11
1	1
2	3
3	6
4	10

Enhanced for Loop (for-each loop)

- JDK 1.5 introduced a new for loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array myList:

```
for (double value: myList)
System.out.println(value);
```

- In general, the syntax is:

```
for (elementType value: arrayRefVar) {
  // Process the value}
```

- You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Reassigning Array References

- An array reference can be assigned to another array of the same type.

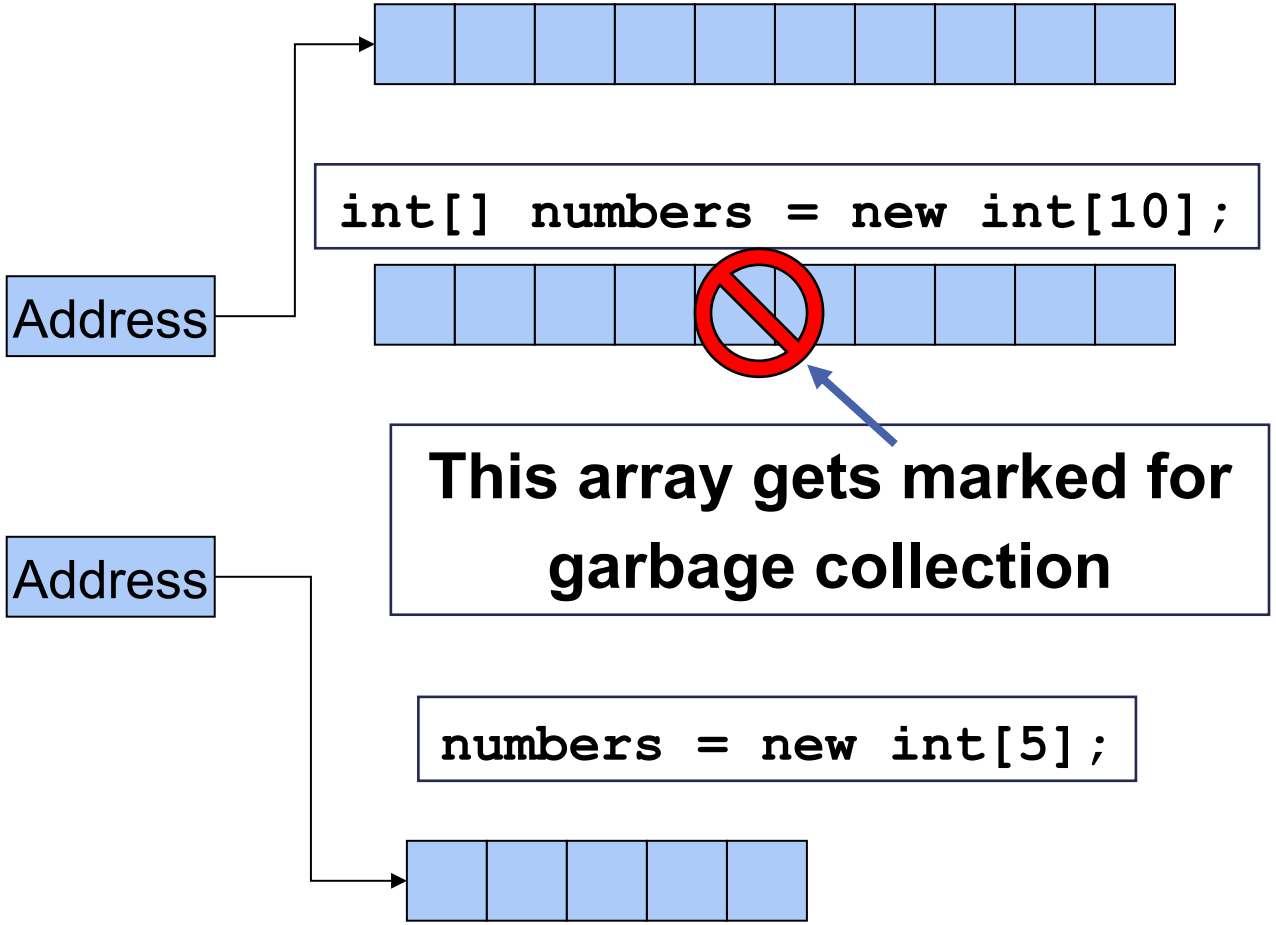
```
// Create an array referenced by the numbers variable.
int[] numbers = new int[10];
// Reassign numbers to a new array.
numbers = new int[5];
```

- If the first (10 element) array no longer has a reference to it, it will be garbage collected.

Reassigning Array References

The numbers variable holds the address of an int array.

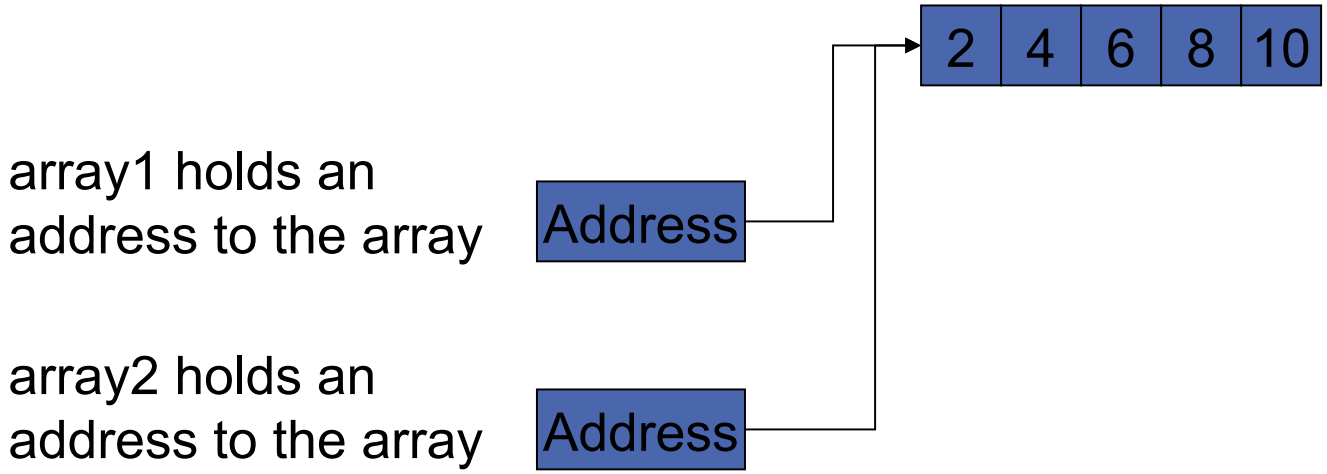
The numbers variable holds the address of an int array.



Copying Arrays

- This is **not** the way to copy an array.

```
int[] array1 = { 2, 4, 6, 8, 10 };
int[] array2 = array1; // This does not copy array1.
```



Copying Arrays

- You cannot copy an array by merely assigning one reference variable to another.
- You need to copy the individual elements of one array to another.

```
int[] firstArray = {5, 10, 15, 20, 25 };
int[] secondArray = new int[5];
for (int i = 0; i < firstArray.length; i+
+)
  secondArray[i] = firstArray[i];
```

- This code copies each element of **`firstArray`** to the corresponding element of **`secondArray`**.

Processing Arrays (Examples)

- See the examples in the text.

1. (Initializing arrays with input values)
2. (Initializing arrays with random values)
3. (Printing arrays)
4. (Summing all elements)
5. (Finding the largest element)
6. (Finding the smallest index of the largest element)

Processing Arrays

- Initializing arrays with input values

```
java.util.Scanner input = new
    java.util.Scanner(System.in);

System.out.print("Enter " + myList.length
    + " values: ");

            for (int i = 0; i <
    myList.length; i++)
                        myList[i] =
    input.nextDouble();
```

Processing Arrays

- Initializing arrays with random values

```
for (int i = 0; i < myList.length;
    i++) {
  myList[i] = Math.random() * 100;
}
```

- Printing arrays

```
for (int i = 0; i < myList.length;
    i++) {
  System.out.print(myList[i] + "
    ");
}
```

Processing Arrays

- Summing all elements

```
double total = 0;
for (int i = 0; i < myList.length; i++)
    {
  total += myList[i];
}
```

- Finding the largest element

```
double max = myList[0];
for (int i = 1; i < myList.length; i++)
    {
  if (myList[i] > max) max = myList[i];
}
```

Declaring Arrays as Formal Parameters to Methods

- General syntax to declare an array as a formal parameter:

```
dataType[] arrayName
```

- Declaring Arrays as Formal Parameters to Methods

```
public static void arraysAsFormalParameter(int[] listA, double[] listB,
int num)
{
    //...
}
```

Declaring Arrays as Formal Parameters to Methods

- Method invoking:

```
int[] intList = new int[10];
double[] doubleNumList = new double[15];
int number;

arraysAsFormalParameter(intList, doubleNumList, number);
```

Comparing Two Arrays Method

- The == operator determines only whether array references point to the same array object.

```
int[] firstArray = { 5, 10, 15, 20, 25 };
int[] secondArray = { 5, 10, 15, 20, 25 };

if (firstArray == secondArray) // This is a mistake.
   System.out.println("The arrays are the same.");
else
   System.out.println("The arrays are not the same.");
```

Comparing Two Arrays Method, cont

```
Public boolean isEqualArrays(int[] firstArray, int[]
secondArray)
{
    if (firstArray.length != secondArray.length)
        return false;
    for (int index = 0; index < firstArray.length;
                        index++)
        if (firstArray[index] != secondArray[index])
            return false;
    return true;
}
if (isEqualArrays(listA, listB))
...
```

Array Processing Methods

- Print :

```
public static void printArray(int[] list, int noOfEle)
 {
    int index;
    for (index = 0; index < noOfEle; index++)
        System.out.print(list[index] + " ");
 }
```

Array Processing Methods

- Summation:

```
public static int sumArray(int[] list, int noOfEle)
 {
    int index;
    int sum = 0;
    for (index = 0; index < noOfEle; index++)
         sum = sum + list[index];
    return sum;
 }
```

Array Processing Methods

- Copying:

```
public static void copyArray(int[] list1, int[] list2,
                                int noOfElements)
{
    int index;
    for (index = 0; index < noOfElements; index++)
         list2[index] = list1[index];
}
 //Main
       int[] firstArray = {5, 10, 15, 20, 25 };
       int[] secondArray = new int[5];
       copyArray(firstArray, secondArray, 5);
```

Array Processing Methods

- Searching:

```
public static int linearSearch(int[] list1, int key)
{

     for (int i = 0; i < list.length; i++)
        {
           if(key==list[i])
                 return I;
        return -1;
        }
}
```

References

- Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos: YouTube channel.
- How to install JDK & NetBeans IDE manual.